



ROOT CAUSE ANALYSIS

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

Aria (VCF Automation) CPU Co-Stop → Soft-Lockup → HA Restart

A 24-vCPU management appliance could not be co-scheduled on a contended nested host — severe CPU co-stop stalled the guest kernel, failed its Kubernetes services, and tripped an HA restart. Root-caused from live metrics; fixed by reducing vCPU 24→16 and verified with before/after co-stop data.

v1.0

VMware Cloud Foundation 9.0.1

RCA — Aria (VCF Automation) CPU Co-Stop → Guest Soft-Lockup → HA Restart

Prepared by: Virtual Control LLC **Date:** July 28, 2026 **Environment:** VMware Cloud Foundation 9.0.1 (nested, single-host homelab) **Affected component:** aria — VCF Automation appliance (on esxi04.lab.local) **Severity:** High (management-plane appliance restarted by HA) — **resolved same day, no data loss**

1. Executive Summary

The `aria` VCF Automation appliance was configured with 24 vCPUs. On the contended nested host it could not co-schedule all 24 vCPUs at once, producing severe CPU co-stop that stalled the guest kernel — soft lockups across multiple CPUs, its Kubernetes control plane (`kube-apiserver`) failed to start, the VM went unresponsive, and vSphere HA restarted it.

- **Not a storage, memory, or application fault.** Memory was healthy (balloon = 0, ~10 GB active of 54 GB) and the CNS/vSAN volumes it backs stayed **Accessible / Compliant** throughout. The cause was purely **CPU co-scheduling**.
- **Confirmed from live metrics:** `cpu.costop.summation` spiked to **42,954 ms** (chronic ~1,800 ms) and `cpu.read.y.summation` to **32,659 ms** — the signature of a too-wide VM on a physically over-subscribed host.
- **Fix:** graceful guest shutdown → reduce **24** → **16 vCPU** → power on. Post-fix co-stop dropped to **avg 46 ms / max 1,526 ms** ($\approx 40\times$ improvement); heartbeat green, stable.
- **Root of the exposure:** VCF Automation ships at a fixed **24 vCPU / 96 GB** per node — a size intended for real hardware, not a nested lab whose ESXi "cores" are slices of one physical workstation shared four ways.

2. Environment Context

Item	Value
Appliance	<code>aria</code> (VCF Automation, Simple/small deployment — 1 node)
Runs on	esxi04.lab.local (nested ESXi)
Physical host	single Dell Precision 7920, cores shared across 4 nested ESXi + the workstation OS
Appliance config (at incident)	24 vCPU / 54 GB RAM
What it backs	the CNS / First-Class-Disk persistent volumes (99 live volumes)

Why a nested lab is uniquely exposed to co-stop. A VM's vCPUs must be **co-scheduled** — the hypervisor tries to run a multi-vCPU VM's vCPUs together. The nested esxi04 presents 24 logical CPUs, but those are backed by a *fraction* of the one physical 7920's cores (shared with three other nested hosts). Asking for 24

vCPUs at once on that physical backing is close to impossible, so the scheduler **stops** some vCPUs waiting for the others — **co-stop** — and from inside the guest that looks like CPUs freezing for tens of seconds.

3. Symptoms (as observed)

- **Guest console** (VM Web Console): repeating kernel messages —
- `watchdog: BUG: soft lockup - CPU#X stuck for 22s/25s!` across multiple CPUs (kubelet, containerd-shim, kube-proxy)
- `[FAILED] Failed to start kube-apiserver.service` (repeating)
- **vSphere**: an **HA restart event** on the appliance (VM was unresponsive → HA reset).
- **After the HA restart**: VM powered on and VMware Tools heartbeat returned **green**, but the K8s control plane was still struggling to come up during boot.

4. Diagnosis — ruled out, then confirmed

Ruled out with live data:

- **Memory pressure — NO.** `balloonMB = 0`, `memActiveMB ≈ 9,953` of 54 GB configured; host esxi04 memory 73%. No ballooning, no swap-driven stall.
- **Host CPU saturation (nested view) — misleading.** esxi04 reported only **37% CPU** — but that is the *nested* host's view. The real contention is at the **physical 7920 layer**, invisible to the nested ESXi. The metric that exposes it is co-stop, not host %CPU.
- **Storage — NO.** The CNS/vSAN Container Volumes it backs were all **Accessible / Compliant** before, during, and after — storage never faulted.

Confirmed root cause — CPU co-stop from an over-wide vCPU count. With the VM at **24 vCPU**, measured on `aria` at the time of the incident:

Counter	Value at incident (24 vCPU)
<code>cpu.costop.summation</code>	spike 42,954 ms , chronic ~1,500–2,000 ms every sample
<code>cpu.ready.summation</code>	spike 32,659 ms

Co-stop of ~43 seconds in a 20-second real-time interval (summed across 24 vCPUs) means the VM spent a large fraction of wall-clock **stopped, waiting for its own vCPUs to co-schedule**. That stall is what the guest kernel reported as "CPU stuck for 22s" soft lockups, which in turn failed `kube-apiserver` / `kubelet` and left the VM unresponsive enough for HA to reset it.

5. Remediation (applied 2026-07-28)

Reduce the appliance from 24 → 16 vCPU. Steps (operator-executed):

1. **Graceful guest shutdown** — Power → *Shut Down Guest OS* (not Power Off), to protect the appliance's etcd / Kubernetes state.
2. **Edit Settings** → **CPU** → **16** (RAM and all else left unchanged at 54 GB).
3. **Power on**; allow the Kubernetes control plane a few minutes to come healthy.

16 vCPUs roughly halves the co-scheduling demand — the hypervisor can line up 16 far more often than 24 on the shared physical cores, so co-stop collapses and the soft lockups stop.

Why this is safe/appropriate here: reducing VCF Automation below its shipped 24 vCPU is a **documented homelab practice** (William Lam / Tom Fojta both edit the powered-off VCFA VM down to 16 or 12 vCPU before power-on). **Memory was left at spec** — memory is genuinely consumed by VCFA (Postgres, Harbor, Istio, the app pods) and must not be cut. Sub-spec vCPU is a lab accommodation, not a Broadcom-supported production configuration.

6. Verification (post-fix, measured live)

Metric	Before (24 vCPU)	After (16 vCPU)	Result
cpu.costop.summation	spike 42,954 ms , chronic ~1,800 ms	avg 46 ms , max 1,526 ms	≈40× lower
cpu.ready.summation	spike 32,659 ms	avg 411 ms, max 6,511 ms	greatly reduced
Power / vCPU	PoweredOn / 24	PoweredOn / 16	resize confirmed
VMware Tools heartbeat	green (recovering)	green (stable)	healthy

Interpretation: co-stop falling from a ~43-second spike / ~1,800 ms chronic down to a **46 ms average** is the direct proof that the co-scheduling stall — the mechanism behind the soft lockups — has been eliminated at this contention level.

7. Prevention & Guidance

- **Right-size wide VMs for the physical backing, not the vendor default.** On this nested lab, keep management appliances at **≤16 vCPU**; 24 vCPU is the ceiling only on real, non-oversubscribed hardware.
- **Monitor co-stop, not just %CPU, for nested guests.** Host %CPU hides physical-layer contention; `cpu.costop` and `cpu.ready` are the true tell. Rule of thumb: sustained co-stop > **~3%** of the interval is a problem; what we saw was far past that.
- **If it recurs:** re-check co-stop first; if still high at 16 vCPU, drop to 12, or reduce contention by not stacking other busy VMs on the same nested host.
- **Watch-after-fix:** confirm `kube-apiserver` / `kubelet` reach and hold healthy inside the appliance after the resize — a clean boot is step one; the K8s services settling is step two.

8. Sources

- Live PowerCLI metrics — `Get-Stat -Stat cpu.costop.summation, cpu.ready.summation` on `aria` (before/after), 2026-07-28.
- Broadcom TechDocs — *VCF Automation Deployment Models / Vertical Scale-Up for Automation*; KB 414551 (VCFA node sizing = 24 vCPU / 96 GB per node).
- William Lam — "Minimal resources for deploying VCF 9.0 in a Lab" (Jun 2025) — documents editing VCFA down to 16/12 vCPU before power-on.
- Tom Fojta — "Deploying VCF 9 in a Minilab" (Jun 2025) — same vCPU-reduction practice.
- VMware — CPU co-scheduling / co-stop background (`esxtop %CSTP` ; performance best-practices for wide VMs).

Document Information

Field	Value
Document ID	VC-RCA-ARIA-COSTOP-20260728
Version	1.0
Issued	July 28, 2026
Author	Virtual Control LLC
Classification	Internal
Platform	VMware Cloud Foundation 9.0.1

Revision History

Version	Date	Notes
1.0	2026-07-28	Initial RCA: 24-vCPU aria co-stop → soft-lockup → HA restart; fixed by 24→16 vCPU; before/after co-stop metrics captured live.

Document End